

AquesTalk2 Mac マニュアル

株式会社 アクエスト
www.a-quest.com

概要

本文書は、音声合成ライブラリ AquesTalk2 Mac をアプリケーションに組み込んで使用するためのプログラミングの方法、注意点を示したものです。

AquesTalk2 は、かな表記の音声記号列から WAV 音声データを生成するライブラリです。

声種の変更は Phont データを差し替えて行います。

評価版ライブラリは、以下の制限があります。製品版ライブラリの入手方法は開発ライセンス証に記載されています。

評価版の制限

「ナ行、マ行」を指定すると、すべて「ヌ」と発声します

また、本ライブラリをアプリケーションに組み込んで使用する際には**使用ライセンス**、配布には**頒布ライセンス**が必要です。ライセンスの種類や購入方法は、弊社サイトのライセンスのページを参照してください。

仕様

ライブラリ形式	dllib 形式共有ライブラリ Arm64 / x86_64
動作環境	Apple シリコン搭載 Mac / Intel Mac
入力データ形式	かな表記音声記号列 (UTF8、UTF16, SJIS)
出力データ形式	WAV フォーマット (8KHz サンプリング, 16bitPCM, モノラル)
声種	14 種
関数 I/F	C 関数呼び出し
マルチスレッド	対応
ライブラリサイズ	約 200KByte (Arm64 / x86_64 FatBinary)
外部依存ライブラリ	/usr/lib/libc++.1.dylib、/usr/lib/libSystem.B.dylib

PHONT ファイル

パッケージの `phont` ディレクトリに、声種を規定する `Phont` ファイルがいくつか含まれています。なお、この中の `ar_f4.phont` は、ライブラリに内蔵のデフォルト `Phont` と同じものです。
ライブラリ呼び出し時に `Phont` ファイルデータを指定することによって、異なる声種で合成が可能となります。

関数 API

AquesTalk2_Synthe_Utf8

AquesTalk2.h

説明	かな表記音声記号列(UTF-8)から音声波形を生成します
構文	<code>unsigned char * AquesTalk2_Synthe_Utf8(const char *koe, int speed, int *size , const void *phontDat)</code>
引数	
<i>koe</i>	音声記号列(UTF8 NULL 終端 BOM は付与しない)を指定
<i>speed</i>	発話速度[%] 50-300 の間で指定 デフォルト:100 値を大きく設定するほど、速くなる
<i>size</i>	生成した音声データのサイズが返る[byte](エラーの場合はエラーコードが返る)
<i>phontDat</i>	phont データの先頭アドレスを指定。内蔵 Phont(F4)を用いるときは0を指定。
戻り値	WAV フォーマットの音声データを返す。 ヒープ領域を関数内部で確保するので、解放は <code>AquesTalk2_FreeWave()</code> で行う。 エラー時は、NULL を返す。このとき <i>size</i> にエラーコードが設定される。

AquesTalk2_Synthe_Utf16

AquesTalk2.h

説明	かな表記音声記号列(UTF-16)から音声波形を生成します
構文	<code>unsigned char * AquesTalk2_Synthe_Utf16(const unsigned short *koe, int speed, int *size , const void *phontDat)</code>
引数	
<i>koe</i>	音声記号列(UTF16 NULL 終端 BOM の付与は任意)を指定
<i>speed</i>	発話速度[%] 50-300 の間で指定 デフォルト:100 値を大きく設定するほど、速くなる
<i>size</i>	生成した音声データのサイズが返る[byte](エラーの場合はエラーコードが返る)
<i>phontDat</i>	phont データの先頭アドレスを指定。内蔵 Phont(F4)を用いるときは0を指定。
戻り値	WAV フォーマットの音声データを返す。

	ヒープ領域を関数内部で確保するので、解放は AquesTalk2_FreeWave()で行う。 エラー時は、NULL を返す。このとき size にエラーコードが設定される。
--	--

AquesTalk2_Synthe

AquesTalk2.h

説明	かな表記音声記号列(SJIS)から音声波形を生成します
構文	unsigned char * AquesTalk2_Synthe (const char *koe, int speed, int *size , const void *phontDat)
引数	
koe	音声記号列 (SJIS NULL 終端)を指定
speed	発話速度[%] 50-300 の間で指定 デフォルト:100 値を大きく設定するほど、速くなる
size	生成した音声データのサイズが返る[byte](エラーの場合はエラーコードが返る)
phontDat	phont データの先頭アドレスを指定。内蔵 Phont(F4)を用いるときは0を指定。
戻り値	WAV フォーマットの音声データを返す。 ヒープ領域を関数内部で確保するので、解放は AquesTalk2_FreeWave()で行う。 エラー時は、NULL を返す。このとき size にエラーコードが設定される。

AquesTalk2_FreeWave

AquesTalk2.h

説明	音声データの領域を開放
構文	void AquesTalk2_FreeWave (unsigned char *wav)
引数	
wav	WAV フォーマットのデータ (AquesTalk2_Synthe()等で生成した音声データ)
戻り値	なし

音声記号列

AquesTalk2 は、かな表記の音声記号列から音声を合成します。漢字を含んだテキスト文字列から音声を合成するときは、別途、言語処理ライブラリ AqKanji2Koe を用いて漢字仮名交じりテキストから音声記号列に変換する必要があります。

音声記号列の詳細は、付属の音声記号列仕様書を参照してください。

エラーコード表

関数が返すエラーコードの内容は、次の通りです。

値	内容
100	その他のエラー
101	メモリ不足
102	音声記号列に未定義の読み記号が指定された
105	音声記号列に未定義の読み記号が指定された
106	音声記号列のタグの指定が正しくない
107	タグの長さが制限を越えている(または[>]が見つからない)
108	タグ内の値の指定が正しくない
200	音声記号列が長すぎる
201	1つのフレーズ中の読み記号が多すぎる
202	音声記号列が長すぎる
203	ヒープメモリ不足
204	音声記号列が長すぎる

サンプルプログラム

HelloAqTalk の 1 つのサンプルプログラム(Xcode プロジェクト)が入っています。
動作確認環境は Xcode V.16.3、macOS 15.4.1 です。

HelloAquesTalk は Swift で記述されたアプリです。エディットボックスに音声記号列を入力し、ボタンの押下でスピーカから合成音声を出力します。なお、Phont ファイルを読み込んで声種を設定しています。



プログラミングガイド

ポイントは、ヘッダファイルのインポートとライブラリの配置・設定です。

ヘッダファイル

ソースコードと同じフォルダにヘッダファイル `AquesTalk2.h` をコピーします。

C++アプリの場合は、`AquesTalk2` の関数を呼び出すソースコードにインクルードします。

```
#include "AuesTalk2.h"
```

Swift の場合は、ブリッジングヘッダーを用意し、その中でヘッダファイルをインポートします。

`HelloAquesTalk-Bridging-Header.h`

```
#import "AuesTalk2.h"
```

Build Setting タブ → Objective-C Bridging Header :

`HelloAquesTalk/HelloAquesTalk-Bridging-Header.h`

ライブラリファイル

プロジェクトに `libAquesTalk2.dylib` をコピーします。コピー場所はプロジェクトディレクトリ(`.xcodeproj` が置かれているフォルダ)がおすすめ。

ビルド時に `dylib` を見つけるための設定

Build Settings タブ → Library Search Paths :

ビルド時の `libAquesTalk2.dylib` のパスを指定（例：`$(PROJECT_DIR)` など）。

Build Phases タブ → Link Binary With Libraries :

`libAquesTalk2.dylib` を指定。

`.dylib` は実行時にも必要です。実行時に `libAquesTalk2.dylib` の配置場所は各種可能ですが、コマンドラインアプリの場合は実行モジュールと同じフォルダに、`.app` の場合は `Frameworks` フォルダ内に配置するのがおすすめです。

```
MyApp.app/  
├── Contents/  
│   └── Frameworks/  
│       └── libAquesTalk2.dylib
```

実行時に `dylib` を見つけるための設定も必要です。

Build Settings タブ → Runpath Search Paths :

実行時の `libAquesTalk2.dylib` のパスを指定します。

実行モジュールと同じフォルダの場合は、

`@executable_path`

Frameworks フォルダ内の場合は、

`@executable_path/../Frameworks`

*評価版は `libAquesTalk2.dylib` の部分を `libAquesTalk2Eva.dylib` に置き換えてください

Tips

■実行時に次のようなエラーが出る場合

```
dyld: Library not loaded: libAquesTalk2.dylib
```

指定したフォルダに `.dylib` があるか、また、下記の設定をチェックします。

Build Settings → Runpath Search Paths に `.dylib` のパスを追加。

設定が正しくても、配置場所によっては `sandbox` で弾かれることがあります。

■ビルド時にアプリバンドル内に `dylib` を自動で入れる設定

Build Phases → Copy Files を追加。

Destination を Frameworks に設定。

`libAquesTalk2.dylib` を追加。

これでビルドされたとき

`MyApp.app/Contents/Frameworks/ libAquesTalk2.dylib`

に自動で配置されます。

■変数の型

`AquesTalk2_Synthe_Utf80`関数の引数や戻り値は `C` の変数型のため、`Swift` などを使用する場合は型変換が必要です。これはサンプルアプリを参考にしてください。

■`AquesTalk2_FreeWav0`

`AquesTalk2_Synthe_Utf80`関数で返された音声データは、使用後に必ず `AquesTalk2_FreeWav0`で解放してください。アプリ側で `free0` を使用して解放するなど異なる `C` ランタイムを使うとハングします。

■声種を切り替える

`Phont` ファイルをアプリの Framework 中の Resource に配置し、合成時に Resource から `Phont` ファイルを読み込むことで動的に声種を切り替えることができます。サンプルプログラム `HelloAquesTalk` を参考にしてください。

文書履歴

日付	版	更新内容
2011/02/06	2.1	新規作成
2025/04/27	2.4	Apple Silicon 対応に伴い全面改訂